

**COMSATS UNIVERSITY ISLAMABAD Abbottabad Campus DEPARTMENT OF COMPUTER
SCIENCE ---**

MID-TERM AI PROJECT REPORT

 AI-BASED CYBER ATTACK PATH SIMULATION SYSTEM

Submitted By: Muhammad Azan Jehangir

Registration Number: CIIT/SP23-BSE-125/ATD

Submitted By: Nouman Dada

Registration Number: CIIT/FA23-BCS-111/ATD

Submitted To: Ms. Zeenat Zulfiqar

Degree Program: Bachelor of Science in Software Engineering (2023–2027)

Contents

Introduction	
ProblemFormulation	
Objectives	
AlgorithmsImplemented	
UninformedSearch	
InformedSearch.....	
LocalSearch.....	
AdversarialSearch	
HeuristicFunction(A*)	
HillClimbingLimitation	
AdversarialSearch(Minimax)	
Alpha-BetaPruning	

ResultsandComparativeAnalysis.	
PerformanceComparisonTable	
0UI/UXDesign.	
DashboardOverview	
BreadthFirstSearch(BFS)	
DepthFirstSearch(DFS)	
UniformCostSearch(UCS)	
A*SearchAlgorithm	
HillClimbing	
Minimax	
Alpha-BetaPrunin	

0.1 Introductory Background

In the domain of modern network security, malicious actors frequently exploit vulnerabilities by laterally traversing a chain of interconnected devices to compromise high-value data repositories. Developing an intimate understanding of these potential target trajectories is paramount for designing resilient infrastructure perimeter defenses.

This academic endeavor addresses the challenge by constructing an intelligent simulation system using Python that models an enterprise network architecture as a mathematical graph. By testing various artificial intelligence search paradigms across this map, the project provides systematic performance data regarding path optimization, processing overhead, and architectural weak links.

0.2 Graph Representation & Model Mapping

The underlying infrastructure layout is mathematically structured as a directed graph network:

- **Vertices/Nodes:** Structural endpoints denoting critical equipment (such as client machines, edge routers, intermediate firewalls, and central SQL databases).
- **Directed Edges:** Legitimate communication paths or access vectors linking adjacent devices.

- **Edge Weights:** Quantitative numerical values representing the security friction, operational risk, or infiltration difficulty associated with moving through that specific pathway.

The simulation tracks an adversarial agent initiating an exploit from a designated **Start Node** with the ultimate goal of establishing a breach at a high-value **Goal Node**.

0.3 Project Objectives

- Represent complex network linkages cleanly via a graph data structure.
- Deploy a suite of operational AI search algorithms from baseline definitions.
- Provide an empirical benchmark comparison across competing pathfinding models.
- Simulate real-world multi-stage lateral threat movements.
- Analyze individual algorithmic trade-offs regarding processing times and route costs.

0.4 Taxonomy of Implemented Search Agents

0.4.1 Unguided Traversal Engines

- **Breadth-First Search (BFS):** A systematic, layer-by-layer exploration strategy managed via a FIFO (First-In, First-Out) tracking structure. It guarantees locating a route containing the fewest hops, though it remains completely blind to step costs along those edges.
- **Depth-First Search (DFS):** An aggressive traversal agent that uses a LIFO (Last-In, First-Out) stack structure to explore down a single branch until it encounters a dead end. While memory-efficient, it frequently settles on highly sub-optimal, high-cost alternative routes.

0.4.2 Guided Traversal Engines

- **Uniform Cost Search (UCS):** A step-cost-aware pathfinding mechanism that utilizes a priority queue to continually branch out from the cheapest cumulative path node. This method guarantees finding the lowest-resistance attack path on a weighted map.
- **A* Search Technique:** An intelligent search variant that prioritizes path evaluation by combining true accumulated distance with forward-looking predictions.

0.4.3 Localized Search Optimization

- **Simple Hill Climbing:** A greedy local search variation that immediately steps into whichever neighboring state reduces the estimated distance remaining, ignoring past path history.

0.4.4 Competitive Adversarial Modeling

- **Minimax Execution Model:** Structures network threat analysis as a two-player, zero-sum game. The threat actor functions as the *Maximizer* (aiming to expand penetration depth), while an automated system defender acts as the *Minimizer* (striving to increase operational costs and block transit paths).
- **Alpha-Beta Pruning:** An optimization layer built over the Minimax tree that cuts off unpromising evaluation branches using mathematical thresholds. This preserves identical optimal outcome paths while minimizing the total number of nodes checked.

0.5 Heuristic Formula Evaluation ($f(n)$)

The directional guidance mechanism for the $f(n)$ search profile relies on the following optimization equation:

$$f(n) = g(n) + h(n)$$

Where the scoring factors are defined as:

- **$g(n)$:** The precise operational cost or risk value accumulated from the starting system to the current vertex n .
- **$h(n)$:** The heuristic forecast estimating the remaining minimum resistance required to reach the final objective target.

This dual tracking behavior allows the engine to skip irrelevant sections of the network graph.

0.6 Topological Limitations of Local Optimization

While Hill Climbing requires minimal processing resources, its greedy local focus leaves it highly vulnerable to structural traps known as **Local Minima**. If the algorithm reaches an intermediate system where all surrounding adjacent nodes present higher weights or poorer heuristic scores than its current position, the execution stalls permanently. It remains unable to discover a global solution even if the true optimal route lies just a single step past that local horizon.

0.7 Game-Theoretic Adversarial Strategy

The simulation views infrastructure protection as an interactive zero-sum game. The Minimax framework maps out multi-step attacking chains while assuming that the network's intrusion prevention protocols will respond with optimal defensive countermeasures at every stage to disrupt or delay the exploit path.

0.8 Optimization via Alpha-Beta Cutoffs

Alpha-Beta Pruning serves as a direct optimization technique for adversarial search trees. By continuously tracking lower and upper bounds of acceptable game states, the engine drops branches that are mathematically proven to be worse than options already checked. This decreases processing latencies without compromising path safety or changing the final tactical route decisions.

0.9 Simulation Telemetry & Analytical Review

Performance evaluations for each tracking script were completed using three baseline operational benchmarks:

- **Total Route Cost:** The total sum of path weights, determining the route's overall efficiency.
- **Expanded States Volume:** The count of infrastructure nodes fully analyzed in memory, indicating memory efficiency.
- **Script Processing Latency (ms):** The actual clock execution time measured in milliseconds.

0.9.1 Script Simulation Comparison Matrix

Evaluated System Agent	Resolved Infrastructure Attack Path	Aggregated Cost	Total Nodes Expanded	Processing Latency (ms)
Breadth-First Search	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	3	1.20
Depth-First Search	A $\rightarrow$ C $\rightarrow$ H $\rightarrow$ E	15	3	0.95

Evaluated System Agent	Resolved Infrastructure Attack Path	Aggregated Cost	Total Nodes Expanded	Processing Latency (ms)
Uniform Cost Search	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	3	1.45
A* Search Algorithm	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	3	1.10
Simple Hill Climbing	A $\rightarrow$ C $\rightarrow$ H	12 (Stuck)	2	

0.10 Visualization Interface Architecture

To provide security personnel with clear visibility into these pathfinding mechanics, an interactive SOC-style (Security Operations Center) visual dashboard panel layout was implemented. The system interface is divided into three distinct functional columns:

- **Control Interface Menu (Left Panel):** Provides a clean selection deck to toggle between search strategies, complete with an *Execute Simulation* command trigger and an active network threat-level alert readout.
- **Topological Canvas Display (Center Panel):** An interactive network visualization space driven by NetworkX and Matplotlib. It renders network nodes, displays connection weights, and uses color-coded paths to trace threat movements across the graph.
- **Analytical Telemetry Feed (Right Panel):** A live data reporting panel displaying critical route logs, including the computed attack sequence, total step weights, node evaluation volumes, execution time metrics, and an automated risk index assessment.

Dashboard Overview



Breadth First Search (BFS)



Depth First Search (DFS)



Uniform Cost Search (UCS)



A* Search Algorithm



Hill Climbing



Minimax



Alpha-Beta Pruning

